

Glossário de Termos — Aula 01

Fundamentos de Banco de Dados Relacionais

Esta aula apresenta os conceitos que sustentam toda a área de banco de dados relacional. Os termos a seguir formam o vocabulário básico que você usará durante todo o curso. Mantenha este glossário por perto como um dicionário de consulta rápida — sempre que encontrar uma palavra técnica nos exercícios ou nas próximas aulas, volte aqui para conferir.

1. Conceitos Fundamentais

Banco de Dados (BD) (Database)

Coleção organizada de dados armazenados de forma estruturada, permitindo consulta, alteração e gerenciamento eficiente. O dado é guardado uma única vez e fica disponível para várias pessoas e sistemas ao mesmo tempo.

Analogia: Pense em uma estante de fichas da biblioteca municipal: cada ficha guarda dados de um livro, e qualquer leitor pode consultar sem precisar mexer no acervo físico.

Dica: Não confundir banco de dados com SGBD. O banco é o conteúdo (os dados); o SGBD é o programa que controla o acesso a esse conteúdo.

SGBD (Sistema de Gerenciamento de Banco de Dados / DBMS)

Programa responsável por criar, manter, acessar e proteger o banco de dados. Ele é a camada entre o usuário e os arquivos físicos: quem solicita dados nunca abre o arquivo diretamente, sempre conversa com o SGBD.

Analogia: É o bibliotecário: você pede o livro pelo título e ele localiza, empresta, devolve e organiza tudo. Você não vai até a prateleira pegar sozinho.

Exemplos de SGBDs: Oracle, MySQL, PostgreSQL, SQL Server, MariaDB, SQLite.

Dica: RDBMS é a sigla em inglês para 'Relational Database Management System' — SGBD Relacional, ou seja, um SGBD que segue o modelo relacional.

Modelo Relacional (Relational Model)

Forma de organizar dados proposta por Edgar Frank Codd em 1970. Nesse modelo, os dados são guardados em tabelas (chamadas relações), e cada tabela representa um tipo de informação do mundo real. Tabelas se conectam por meio de chaves.

Analogia: Como uma planilha do Excel: linhas representam registros e colunas representam características. Várias planilhas podem se ligar entre si por um código em comum.

Dica: Antes do modelo relacional, os bancos eram hierárquicos ou em rede — bem mais engessados. O modelo relacional venceu porque é simples e flexível.

SQL (Structured Query Language)

Linguagem padrão para se comunicar com bancos de dados relacionais. Permite consultar, inserir, alterar, remover e estruturar dados. É reconhecida pela ANSI e pela ISO como padrão internacional.

Analogia: É o idioma oficial usado para conversar com qualquer SGBD relacional — assim como o inglês funciona como idioma comum em aeroportos do mundo todo.

```
SELECT nome FROM funcionarios
WHERE departamento = 'TI';
```

Dica: SQL não é igual em todos os SGBDs — cada banco tem pequenas variações (dialetos), mas o núcleo da linguagem é o mesmo em todos.

2. Modelo Entidade-Relacionamento (MER)

Modelo Entidade-Relacionamento (MER / DER / Modelo ER)

Representação gráfica usada para planejar um banco de dados antes de criá-lo. Mostra quais informações serão armazenadas (entidades), o que descreve cada uma (atributos) e como elas se conectam (relacionamentos).

Analogia: É a planta baixa de uma casa: antes de levantar parede, o engenheiro desenha onde vai cada cômodo e como eles se ligam.

Dica: MER é a teoria; DER (Diagrama Entidade-Relacionamento) é o desenho propriamente dito.

Entidade (Entity)

Qualquer coisa relevante do mundo real sobre a qual queremos guardar informações. Pode ser pessoa, objeto, lugar, evento ou conceito. No DER, é desenhada como um retângulo de cantos arredondados, com nome em letras maiúsculas.

Analogia: É como uma categoria do estoque de uma loja: Clientes, Produtos, Vendas. Cada categoria guarda informações de um tipo só.

Entidades comuns:

- FUNCIONARIO
- DEPARTAMENTO
- CLIENTE
- PEDIDO

Atributo (Attribute)

Característica que descreve uma entidade. Cada entidade tem vários atributos, e cada um pode ser obrigatório (marcado com asterisco *) ou opcional (marcado com a letra o).

Analogia: Em uma ficha cadastral, os atributos são os campos a preencher: nome, CPF, telefone, e-mail. Alguns são obrigatórios, outros podem ficar em branco.

```
FUNCIONARIO
  #* numero    (obrigatorio, identificador)
  * nome       (obrigatorio)
  o cargo      (opcional)
```

Dica: Atributo obrigatório nunca pode ser nulo; atributo opcional pode aceitar valor nulo (NULL).

Relacionamento (Relationship)

Associação entre duas entidades, indicando como elas se ligam. No DER, é representado por uma linha que conecta as entidades, com um nome (verbo) que descreve a ligação.

Analogia: É a ponte entre duas ilhas: cada ilha tem suas casas (registros), e a ponte indica que uma ilha depende ou interage com a outra.

```
FUNCIONARIO --- atribuido a ---> DEPARTAMENTO
DEPARTAMENTO --- composto de ---> FUNCIONARIO
```

Cardinalidade (Cardinality / Grau)

Indica quantas ocorrências de uma entidade podem estar associadas a outra. As três cardinalidades clássicas são: 1 para 1 (1:1), 1 para muitos (1:N) e muitos para muitos (N:N).

Analogia: Um pai pode ter vários filhos, mas cada filho tem apenas um pai biológico — isso é uma relação 1:N entre PAI e FILHO.

```
DEPARTAMENTO 1 ---- N FUNCIONARIO
ALUNO        N ---- N DISCIPLINA
```

Dica: Relacionamentos N:N normalmente exigem uma tabela intermediária quando viram tabelas no banco — veremos isso em aulas futuras.

Opcionalidade (Optionality)

Indica se a participação de uma entidade no relacionamento é obrigatória (deve ser) ou opcional (pode ser). No DER, linha sólida = obrigatório; linha tracejada = opcional.

Analogia: Todo casamento precisa de duas pessoas (obrigatório), mas nem toda pessoa precisa estar casada (opcional).

Identificador Único (UID / Unique Identifier)

Atributo (ou combinação de atributos) que distingue uma ocorrência de outra dentro de uma mesma entidade. Não existem dois registros com o mesmo UID. No DER, é marcado com o símbolo de tralha (#).

Analogia: É o CPF de cada pessoa: não existem dois brasileiros com o mesmo CPF, ele serve para identificar cada um de forma exclusiva.

Dica: O UID no modelo conceitual vira a Chave Primária quando o modelo é convertido em tabelas reais no banco.

3. Estrutura das Tabelas

Tabela (Relação / Table)

Estrutura bidimensional (linhas e colunas) que guarda os dados de uma entidade. É a unidade básica de armazenamento em um banco de dados relacional. Cada tabela deve representar exatamente uma entidade.

Analogia: Uma planilha do Excel ou uma grade do caderno: linhas embaixo, colunas ao lado, valores nas células.

```
Tabela: FUNCIONARIOS
+---+-----+-----+
| id | nome  | salario |
```

```

+-----+-----+-----+
| 1 | Ana | 4500.00 |
| 2 | Bruno | 3800.00 |
+-----+-----+-----+

```

Linha (Registro / Tupla / Row)

Cada conjunto horizontal de valores em uma tabela. Representa uma ocorrência completa da entidade — todos os dados de UM funcionário, UM produto, UM cliente. A ordem das linhas não importa para o banco.

Analogia: Cada ficha individual no fichário do consultório: uma ficha por paciente, com todas as informações dele em uma só linha de dados.

Dica: Duas linhas nunca podem ser totalmente iguais em uma tabela bem projetada — sempre haverá ao menos uma coluna (a PK) que diferencia as duas.

Coluna (Atributo / Campo / Column)

Cada divisão vertical de uma tabela. Representa um tipo específico de informação, com um nome e um tipo de dado fixo (texto, número, data...). A ordem das colunas no armazenamento também não importa.

Analogia: Os campos de um formulário em papel: 'Nome', 'CPF', 'Telefone' — cada campo aceita um tipo específico de informação.

Campo (Field)

Interseção entre uma linha e uma coluna. É a célula que guarda um único valor. Cada campo armazena apenas UM valor — não é possível guardar dois números de telefone no mesmo campo, por exemplo.

Analogia: É como uma gaveta de organizador: cada gaveta tem um único item. Se você precisa guardar dois itens, abre outra gaveta.

NULL (Valor Nulo)

Marca que indica ausência de valor em um campo. Não é zero, não é vazio, não é espaço em branco — é a representação de 'desconhecido' ou 'não se aplica'. Apenas colunas opcionais podem receber NULL.

Analogia: É como deixar um campo do formulário sem preencher: você não escreveu nada porque a informação ainda não existe ou não se aplica ao seu caso.

```
comissao_pct = NULL    -- funcionario sem comissao
```

Dica: NULL não é igual a zero e nem a string vazia (''). Em SQL, comparações com NULL exigem 'IS NULL' ou 'IS NOT NULL' — usar '= NULL' não funciona.

4. Chaves (PK e FK)

Chave Primária (PK / Primary Key)

Coluna (ou conjunto de colunas) que identifica de forma única cada linha de uma tabela. Não pode ter valores repetidos e não pode ser nula. Toda tabela bem projetada tem uma chave primária.

Analogia: É o número da matrícula do aluno: cada aluno tem um número diferente, e esse número é o que identifica oficialmente cada estudante na escola.

```
Tabela: FUNCIONARIOS
  id_funcionario <- chave primaria
  nome
  salario
  email
```

Dica: O valor da PK normalmente não muda ao longo do tempo. Mudar PK quebra ligações com outras tabelas — evite fazer isso.

Chave Estrangeira (FK / Foreign Key)

Coluna em uma tabela que faz referência à chave primária de outra tabela (ou da mesma tabela). Serve para conectar registros entre tabelas diferentes, criando o relacionamento físico no banco.

Analogia: É como o RG do responsável anotado na ficha do aluno: aquela ficha aponta para uma pessoa cadastrada em outro lugar.

```
FUNCIONARIOS
  id_funcionario (PK)
  nome
  id_departamento (FK) ----+
                               |
DEPARTAMENTOS
  id_departamento (PK) <----+
  nome_departamento
```

Dica: Uma chave estrangeira deve corresponder a uma chave primária existente — ou ser NULL. Não pode apontar para nada inexistente.

Integridade Referencial (Referential Integrity)

Regra que garante que toda chave estrangeira aponte para um valor válido na tabela de origem. O SGBD recusa inserir um pedido vinculado a um cliente que não existe, por exemplo.

Analogia: O cartório não registra um casamento com uma pessoa fictícia. Se o nome do noivo não existe na base de pessoas, o casamento é recusado.

Dica: Quando você tenta apagar um cliente que tem pedidos vinculados, o banco bloqueia a operação — é a integridade referencial em ação.

5. Categorias de Instruções SQL

DDL (Data Definition Language)

Subconjunto da linguagem SQL responsável por definir e alterar a estrutura do banco — criar, modificar e remover tabelas, índices, colunas. É a parte da linguagem que mexe na 'arquitetura'.

Analogia: É o pedreiro: levanta paredes, derruba paredes, muda o formato da casa. Não guarda nem retira móveis — só altera a estrutura.

```
CREATE -- cria estrutura
ALTER -- altera estrutura
DROP -- remove estrutura
TRUNCATE -- esvazia tabela
RENAME -- renomeia objeto
```

DML (Data Manipulation Language)

Subconjunto da SQL voltado para manipular os dados dentro das tabelas: consultar, inserir, atualizar e remover linhas. É a parte mais usada no dia a dia por quem trabalha com banco de dados.

Analogia: É o morador da casa: coloca móveis, troca móveis, retira móveis, olha o que tem dentro. Não mexe na estrutura do imóvel.

```
SELECT -- consulta dados
INSERT -- adiciona linhas
UPDATE -- altera linhas
DELETE -- remove linhas
MERGE -- combina insert + update
```

DCL (Data Control Language)

Subconjunto da SQL que controla permissões de acesso ao banco — quem pode ler uma tabela, quem pode alterar, quem pode apagar. Usada por administradores e responsáveis pela segurança dos dados.

Analogia: É o porteiro do prédio: decide quem entra, quem sobe até cada andar, quem tem chave de cada apartamento.

```
GRANT -- concede permissao
REVOKE -- retira permissao
```

Dica: Em equipes pequenas, normalmente o administrador do banco cuida de DCL — desenvolvedores raramente usam essas instruções.

TCL (Transaction Control Language)

Subconjunto da SQL para controlar transações — confirmar ou desfazer um conjunto de operações DML como se fossem uma só. Garante que mudanças complexas aconteçam de forma completa ou não aconteçam de jeito nenhum.

Analogia: É como o botão 'Confirmar' no caixa eletrônico: se algo dá errado no meio do saque, a operação inteira é cancelada — o saldo não pode ficar inconsistente.

```
COMMIT -- confirma alteracoes
ROLLBACK -- desfaz alteracoes
SAVEPOINT -- marca ponto de retorno
```

6. Outros Conceitos Importantes

ANSI / ISO (Órgãos de padronização)

ANSI (American National Standards Institute) e ISO (International Standards Organization) são as entidades que definem o padrão internacional da linguagem SQL. Tudo o que está no padrão funciona em qualquer SGBD.

Analogia: É como o sistema métrico: o metro tem o mesmo tamanho no Brasil e no Japão porque existe uma entidade internacional que define a medida.

Dicionário de Dados (Data Dictionary)

Conjunto de tabelas internas mantidas pelo próprio SGBD com informações sobre o banco: quais tabelas existem, quais colunas têm cada tabela, quais usuários estão cadastrados. É o 'catálogo' do banco.

Analogia: É o sumário e o índice remissivo de um livro técnico: você consulta primeiro para saber onde está cada assunto.

Transação (Transaction)

Conjunto lógico de uma ou mais instruções DML que precisam ser executadas como uma unidade indivisível. Se uma parte falha, todas são desfeitas — princípio ATÔMICO das transações.

Analogia: Transferência bancária: debitar de uma conta e creditar em outra precisam acontecer juntos. Se só o débito ocorrer, o dinheiro 'somou'.

```
-- Início implícito
UPDATE conta SET saldo = saldo - 100 WHERE id = 1;
UPDATE conta SET saldo = saldo + 100 WHERE id = 2;
COMMIT; -- confirma as duas juntas
```